

Guía de Laboratorio · Sesión 2

Tu primera imagen — construida, publicada y endurecida

Curso 101-EAN · Gestión de Contenedores en Entornos de Nube Híbrida · Duración: 1 hora Plataforma: **Play with Docker** (navegador, gratis)

Hoy escribes tu primer Dockerfile, construyes TU imagen, la publicas en Docker Hub y compruebas la magia: otra máquina la corre con un solo comando. Cierras endureciéndola: tu contenedor sin root. Entregable: **la URL de tu imagen en el Hub**.

0. Requisitos (5 min)

1. Tu cuenta de **Docker Hub** de la S1 (usuario y contraseña **a la mano** — hoy sí la usas).
2. Entra a labs.play-with-docker.com → Login → Start → + **ADD NEW INSTANCE**.

⌚ Recuerda: la sesión de PWD se borra al expirar. Tu imagen sobrevivirá porque quedará publicada en Docker Hub — de eso se trata la sesión de hoy.

Parte 1 · Construye tu imagen (25 min)

1.1 — Crea tu carpeta de trabajo y tu página:

```
mkdir mi-web && cd mi-web
cat > index.html <<'EOF'
<!DOCTYPE html>
<html>
<head><meta charset="utf-8"><title>Mi primera imagen</title></head>
<body style="font-family: sans-serif; text-align:center; margin-top:10%">
  <h1>🐳 ¡Hola! Soy TU-NOMBRE</h1>
  <p>Esta página viaja empacada en mi primera imagen Docker.</p>
  <p><b>101-EAN · Gestión de Contenedores · Colpensiones</b></p>
</body>
</html>
EOF
```

Edita el `TU-NOMBRE` (puedes usar `vi index.html` o volver a correr el `cat` con tu nombre).

1.2 — Escribe tu primer Dockerfile:

```
cat > Dockerfile <<'EOF'
FROM nginx:alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 80
EOF
```

Dos instrucciones tuyas — la base oficial hace el resto. 🗝️ `nginx:alpine` = oficial y mínima.

1.3 — Construye y corre:

```
docker build -t mi-web:1.0 .
docker images          # ahí está: mi-web 1.0
docker run -d -p 8080:80 --name web1 mi-web:1.0
```

Clic en el enlace **8080** → tu página, con tu nombre, servida por TU imagen.

1.4 — El caché en acción. Cambia algo del `index.html` (el título, un emoji) y:

```
docker build -t mi-web:1.1 .
```

👉 **Para la bitácora:** compara la salida de los dos builds. ¿Qué capas dicen `CACHED`? ¿Por qué el segundo fue casi instantáneo?

Parte 2 · Publícala en Docker Hub (20 min)

2.1 — Etiqueta con tu usuario (tu imagen vive bajo tu nombre en el Hub):

```
docker tag mi-web:1.1 TU-USUARIO/mi-web:1.1    # ← tu usuario de Docker Hub
```

2.2 — Auténticate y publica:

```
docker login          # usuario y contraseña del Hub
docker push TU-USUARIO/mi-web:1.1
```

2.3 — Vela en el mundo: abre <https://hub.docker.com/r/TU-USUARIO/mi-web> — tu imagen, pública, con su tag.

2.4 — La prueba reina. Clic en + **ADD NEW INSTANCE** (¡una máquina NUEVA que jamás vio tu código!) y en esa terminal:

```
docker run -d -p 8080:80 TU-USUARIO/mi-web:1.1
```

Abre el 8080 de ESA instancia: tu página. La imagen viajó por el registro.

🔥 **Para la bitácora:** captura de tu página en el Hub + captura del run en la instancia nueva. ¿Qué pasos hizo Docker solo en la máquina nueva?

Parte 3 • 🛡️ Endurécela (15 min)

3.1 — ¿Quién corre tu contenedor? En la instancia original:

```
docker exec web1 whoami          # → root 🤖
```

Root dentro del contenedor: si comprometen la web, el atacante entra con privilegios.

3.2 — La versión endurecida. nginx publica una variante **sin privilegios** (publicador verificado — compruébalo en el Hub: `nginxinc/nginx-unprivileged`):

```
cat > Dockerfile <<'EOF'
FROM nginxinc/nginx-unprivileged:alpine
COPY index.html /usr/share/nginx/html/
EXPOSE 8080
EOF
docker build -t mi-web:1.2-segura .
docker run -d -p 9090:8080 --name web-segura mi-web:1.2-segura
docker exec web-segura whoami    # → nginx ✅ (no root)
```

(Nota el cambio: esta variante escucha en el **8080** interno porque los puertos bajos requieren root — por eso el `-p 9090:8080`.)

3.3 — Las capas no mienten:

```
docker history mi-web:1.2-segura
```

Ahí está tu receta, capa por capa — visible para cualquiera que tenga la imagen. Por eso: **cero secretos en la imagen, nunca.**

🔥 **Para la bitácora:** ¿qué devolvió `whoami` en cada versión? ¿Por qué a una entidad financiera le importa esa diferencia?

3.4 — Limpieza: `docker rm -f web1 web-segura` y verifica con `docker ps -a`.

Checklist de la sesión

☐ Escribí mi Dockerfile y construí mi-web:1.0.

- ☐ Vi el caché de capas en el build de la 1.1.
- ☐ Publiqué mi imagen bajo mi usuario en Docker Hub.
- ☐ La corré en una instancia nueva con un solo comando.
- ☐ Comprobé root vs no-root con whoami y construí la versión endurecida.
- ☐ Revisé las capas con docker history.

Entregable de la S2

1. La URL de tu imagen: `https://hub.docker.com/r/TU-USUARIO/mi-web`
2. Captura del `docker run` funcionando en la **instancia nueva**.
3. Bitácora con las 3 reflexiones de las partes 1, 2 y 3.

Próxima sesión: tu web no vive sola — la conectas con otros contenedores en **redes aisladas**: una app multi-contenedor de verdad. 🕸